

**FAST – National University of Computer and Emerging
Sciences**

Department of Computer Science, Islamabad Campus

University
Logo
Here

VehicleWalay

**AI-Powered Vehicle Discovery &
Recommendation Platform**

FYP-1 Midterm Report • Fall 2025

Submitted By Bilal Aleem (22I-1866)

Awais Khaleeq (22I-2039)

Supervisor Adil Majeed

Co-Supervisor Mr. Ahmad Raza

Programme BS Computer Science

Semester 7th Semester, Fall 2025

Contents

1	Introduction	1
1.1	Background	1
1.2	Literature Review	1
2	Project Vision	3
2.1	Problem Statement	3
2.2	Business Opportunity	3
2.3	Objectives	4
2.4	Project Scope	4
2.5	Constraints	5
2.6	Stakeholder Description	5
2.6.1	Stakeholders Summary	6
2.6.2	Key High-Level Goals and Problems of Stakeholders . .	6
3	Software Requirements Specification	7
3.1	List of Features	8
3.2	Functional Requirements	8
3.2.1	User Authentication	9
3.2.2	Vehicle Search and NLU	9
3.2.3	Recommendations	9
3.2.4	Data Ingestion and Fake Detection	9
3.2.5	Alerts	10
3.2.6	Meeting Automation	10
3.3	Non-Functional Requirements	10
3.3.1	Performance	10
3.3.2	Reliability	10
3.3.3	Security	11

3.3.4	Usability	11
3.3.5	Scalability	11
3.4	Use Case Diagram	11
4	Software Design	13
4.1	Design Methodology	13
4.2	Architectural Style	13
4.3	Architectural Diagram	14
4.4	Process Flow — Activity Diagram	15
4.5	Class Diagram	16
4.6	Sequence Diagram — System Level	17
4.7	State Diagram	18
4.8	Data Flow Diagram — Context Level (DFD Level 0)	20
4.9	Data Flow Diagram — Level 1	21
4.10	Entity-Relationship Diagram	22
5	Iteration Plan	24
6	Ethical & Legal Considerations	25
7	Comparison with Existing Systems	26
	References	28

List of Figures

3.1	Use Case Diagram — VehicleWalay	12
4.1	Architectural Diagram — VehicleWalay End-to-End System . .	15
4.2	Activity Diagram — Vehicle Search and Recommendation Flow	16
4.3	Class Diagram — VehicleWalay Domain Model (OOD)	17
4.4	Sequence Diagram — UC-02: NLU Vehicle Search (System Level)	18
4.5	State Diagram — Vehicle Listing Lifecycle	19
4.6	Data Flow Diagram — Context Level (Level 0)	20
4.7	Data Flow Diagram — Level 1 Process Decomposition	21
4.8	Entity-Relationship Diagram — VehicleWalay Data Model . . .	22

List of Tables

2.1	Stakeholder Summary	6
2.2	Stakeholder Goals and Problems	6
3.1	Feature List	8
5.1	Iteration Plan	24
7.1	Feature Comparison: VehicleWalay vs. Existing Platforms . . .	27

Chapter 1

Introduction

1.1 Background

Pakistan’s used-vehicle market is among the most active in South Asia, yet remains fragmented, opaque, and inaccessible to a significant portion of its population. Vehicle seekers today navigate a patchwork of platforms — PakWheels, OLX Motors, dealership websites, Facebook groups, and WhatsApp forwards — with no unified, intelligent interface capable of synthesising listings, evaluating their authenticity, or guiding buyers through the complexity of the purchase process.

VehicleWalay is a final-year project that addresses this gap through the design and implementation of an AI-powered vehicle discovery and recommendation platform tailored specifically for the Pakistani market. At its core, the system deploys a Retrieval-Augmented Generation (RAG) chatbot that processes natural-language and voice queries in both English and Roman Urdu, retrieves live vehicle listings, detects fraudulent entries, generates AI-authored walkthrough content, and automates buyer–dealer scheduling — all within a single, coherent interface.

1.2 Literature Review

The problem of intelligent vehicle discovery sits at the intersection of three active research areas: natural language processing applied to e-commerce search, recommender systems for high-involvement purchases, and fraud detection in

peer-to-peer marketplaces.

RAG and Semantic Search

Devlin et al. (2018) introduced BERT-based contextual embeddings that have since become the foundation for semantic search in e-commerce. Karpukhin et al. (2020) formalised Dense Passage Retrieval (DPR), and Lewis et al. (2020) established the RAG paradigm that this project adopts directly: a retriever surfaces relevant vehicle listings from a vector store, and a generative model synthesises a conversational response.

Vehicle Recommendation

Content-based filtering using vehicle metadata (make, model, year, fuel type, city) outperforms collaborative filtering in sparse, cold-start environments typical of regional marketplaces (Zhang et al., 2019). Sentence Transformers (Reimers & Gurevych, 2019) operating on MiniLM provide CPU-friendly embedding generation suitable for FYP deployment without GPU infrastructure.

Fake Listing Detection

Studies on OLX and Craigslist data show that listings priced more than two standard deviations below the market median, combined with vague or inconsistent descriptions, are strong indicators of fraudulent intent (Chou et al., 2021). VehicleWalay incorporates a rule-augmented embedding-based classifier to flag such listings at ingestion time.

Multilingual NLU

The Roman Urdu requirement is addressed through prompt engineering and few-shot examples fed to the language model, a technique validated by Brown et al. (2020) in GPT-3 and confirmed effective for low-resource languages by Winata et al. (2021).

Chapter 2

Project Vision

2.1 Problem Statement

Finding vehicles in Pakistan is an inefficient, frustrating, and often deceptive experience. The core problems are:

- **Fragmentation:** Buyers must simultaneously monitor PakWheels, OLX Motors, Facebook Groups, individual dealer websites, and informal WhatsApp broadcast lists to achieve reasonable market coverage.
- **Listing Quality:** A substantial proportion of active listings are outdated, already sold, priced incorrectly, or deliberately fraudulent.
- **Language Exclusion:** All major platforms assume English literacy, alienating the large segment of the population that searches exclusively in Urdu or Roman Urdu.
- **Absence of Intelligence:** No platform offers intent-aware search (e.g., “*fuel-efficient family sedan under 25 lakh for Lahore city driving*”), personalised recommendations, or proactive price-change alerts.
- **Communication Friction:** Buyers and dealers waste significant time on repetitive, low-value telephone enquiries that could be automated.

2.2 Business Opportunity

Pakistan’s used-vehicle market transacts over 500,000 units annually, with PakWheels alone reporting more than 10 million monthly active users. No existing platform in the region offers AI-driven search, multilingual support, or end-to-

end buyer–dealer automation. VehicleWalay occupies this white space, with a clear path to monetisation through premium dealer subscriptions, featured listings, and lead-generation fees once the platform reaches production scale.

2.3 Objectives

1. Design and implement a RAG-based conversational chatbot capable of processing vehicle discovery queries in English and Roman Urdu.
2. Build a real-time web scraping pipeline that continuously ingests and de-duplicates listings from PakWheels and OLX Motors.
3. Implement an embedding-based fake-listing detection classifier achieving $> 85\%$ precision on a labelled test set.
4. Develop a content-based vehicle recommendation engine using Sentence Transformers (MiniLM) without GPU dependency.
5. Deliver an automated buyer–dealer scheduling agent that confirms test-drive appointments via chatbot interaction.
6. Generate AI-authored video-style walkthrough summaries for individual vehicle listings.
7. Provide a multilingual interface supporting English and Roman Urdu voice and text input.

2.4 Project Scope

VehicleWalay is scoped as a web-based platform serving the Pakistani vehicle market. The system encompasses:

- A Next.js 14 frontend with an embedded, customisable chatbot UI and a vehicle-card browsing interface.
- A FastAPI backend exposing RESTful endpoints for authentication, vehicle search, recommendations, alerts, and meeting scheduling.
- An AI layer comprising LangChain orchestration, MiniLM sentence embed-

dings, FAISS vector store, and an LLM (OpenAI GPT-4o or Llama 3 via OpenRouter).

- A real-time data ingestion pipeline scraping PakWheels and OLX Motors, with per-listing fake-score computation on ingestion.
- A MongoDB primary database for listings, users, and alerts, with Redis for session caching.

Out of scope for FYP-1: payment processing, mobile native applications, and integration with government vehicle registration APIs.

2.5 Constraints

- No GPU infrastructure available; all model inference must be CPU-compatible or API-based.
- Data collection is performed via real-time web scraping of publicly accessible listing pages (PakWheels, OLX Motors); no private or user-submitted personal data is stored.
- The platform complies with Pakistani data protection norms; users are informed of notification opt-ins before consent is recorded.
- Development is distributed between two team members working approximately one hour per day alongside coursework.
- Deployment is constrained to free-tier or low-cost cloud infrastructure (Render / Railway / Vercel) for FYP demonstration.

2.6 Stakeholder Description

2.6.1 Stakeholders Summary

Table 2.1. Stakeholder Summary

Stakeholder	Role	Interest
Vehicle Buyer	Primary End User	Find the right vehicle quickly, avoid scams, receive price alerts
Vehicle Dealer	Secondary End User	Maximise qualified enquiries, reduce repetitive calls, access analytics
Project Supervisor	Academic Evaluator	Technically sound implementation, adherence to FYP guidelines
FYP Committee	Examiner	Complete documentation, correct diagrams, working prototype
Development Team	Implementers	Build a technically interesting, demonstrable system

2.6.2 Key High-Level Goals and Problems of Stakeholders

Table 2.2. Stakeholder Goals and Problems

Stakeholder	Goal	Problem Today
Buyer	Find a verified, affordable vehicle fast	Fragmented search, fake listings, language barriers
Dealer	Reach serious buyers efficiently	Buried in casual enquiries, no qualified lead filtering
Supervisor	Evaluate a complete, well-documented FYP	Incomplete SRS and design artefacts from students

Chapter 3

Software Requirements Specification

3.1 List of Features

Table 3.1. Feature List

ID	Feature	Description
F-01	NLU Vehicle Search	Conversational search in English and Roman Urdu
F-02	AI Recommendation Engine	Personalised suggestions via content-based RAG filtering
F-03	Real-Time Listing Ingestion	Continuous scraping of PakWheels and OLX Motors
F-04	Fake Listing Detection	Anomaly scoring on price and description at ingestion
F-05	Smart Alerts	Push notifications: price change, sold, similar listing
F-06	Multilingual Voice & Text	Web Speech API for voice; Roman Urdu via LLM prompt tuning
F-07	AI Video Walkthrough	Auto-generated narrated slideshow per vehicle listing
F-08	Meeting Automation	Chatbot-driven test-drive scheduling
F-09	Market Insights	Trending vehicles by city, brand, and price range
F-10	User History & Recall	Persistent search history and preference personalisation

3.2 Functional Requirements

3.2.1 User Authentication

- **FR-01:** The system shall allow users to register with email and password.
- **FR-02:** The system shall authenticate users via JWT-signed tokens with a configurable expiry period.
- **FR-03:** The system shall support role-based access control with two roles: *Buyer* and *Dealer*.

3.2.2 Vehicle Search and NLU

- **FR-04:** The system shall accept free-text queries and extract intent, vehicle attributes (make, model, budget, city, fuel type), and language (English / Roman Urdu).
- **FR-05:** The system shall return a ranked list of at most ten matching listings per query, ordered by relevance score.
- **FR-06:** The system shall support voice input via the Web Speech API and convert speech to text before NLU processing.

3.2.3 Recommendations

- **FR-07:** The system shall generate personalised recommendations based on a user's last five searches and explicit preference settings.
- **FR-08:** Recommendations shall be computed using cosine similarity over MiniLM sentence embeddings without requiring GPU inference.

3.2.4 Data Ingestion and Fake Detection

- **FR-09:** The scraping pipeline shall poll PakWheels and OLX Motors every six hours, extracting: vehicle title, price, city, make, model, year, mileage, seller type, listing URL, and listing date.
- **FR-10:** Each ingested listing shall receive a *fake-score* (0–1) computed from: price *z*-score relative to make/model median, description length, listing age, and image count.

- **FR-11:** Listings with a fake-score ≥ 0.75 shall be flagged and suppressed from default search results, but remain accessible via an explicit “show flagged” filter.

3.2.5 Alerts

- **FR-12:** A buyer shall be able to add up to 20 vehicles to a watchlist.
- **FR-13:** The system shall send an in-app and email notification within 15 minutes of a price change, sold-status update, or appearance of a similar listing.

3.2.6 Meeting Automation

- **FR-14:** A buyer shall be able to initiate a test-drive request from any vehicle listing card within the chatbot interface.
- **FR-15:** The system shall present three available time slots fetched from the dealer’s availability calendar and confirm the appointment upon buyer selection.

3.3 Non-Functional Requirements

3.3.1 Performance

- **NFR-01:** The chatbot shall return a response within 3 seconds for 95% of queries under a load of 100 concurrent users.
- **NFR-02:** The RAG retrieval step shall complete within 500 ms for a vector store of up to 500,000 documents.

3.3.2 Reliability

- **NFR-03:** The system shall maintain 99% uptime during demonstration periods via Docker-containerised deployment with automatic restart policies.
- **NFR-04:** The scraping pipeline shall implement exponential back-off and retry logic, completing $\geq 98\%$ of scheduled polling runs successfully.

3.3.3 Security

- **NFR-05:** All API endpoints except /auth/register and /auth/login shall require a valid JWT token in the Authorization header.
- **NFR-06:** User passwords shall be stored as bcrypt hashes (cost factor 12); no plaintext credentials shall appear in logs.
- **NFR-07:** The system shall scrape only publicly accessible listing pages. No personal user data from third-party platforms is stored or transmitted.

3.3.4 Usability

- **NFR-08:** The chatbot UI shall render correctly on Chrome, Firefox, and Safari on desktop and mobile viewports.
- **NFR-09:** A first-time user shall complete a vehicle search query within 30 seconds of landing on the chatbot page, without reading documentation.

3.3.5 Scalability

- **NFR-10:** The FastAPI backend shall be stateless so that horizontal scaling is achievable by adding container replicas behind a load balancer.
- **NFR-11:** MongoDB collections shall include indexes on city, make, model, price, and status fields to maintain sub-50 ms query latency at one million documents.

3.4 Use Case Diagram

The Use Case Diagram (Figure 3.1) captures all interactions between the primary actors — Buyer, Dealer, and the automated System Scheduler — and the VehicleWalay platform.

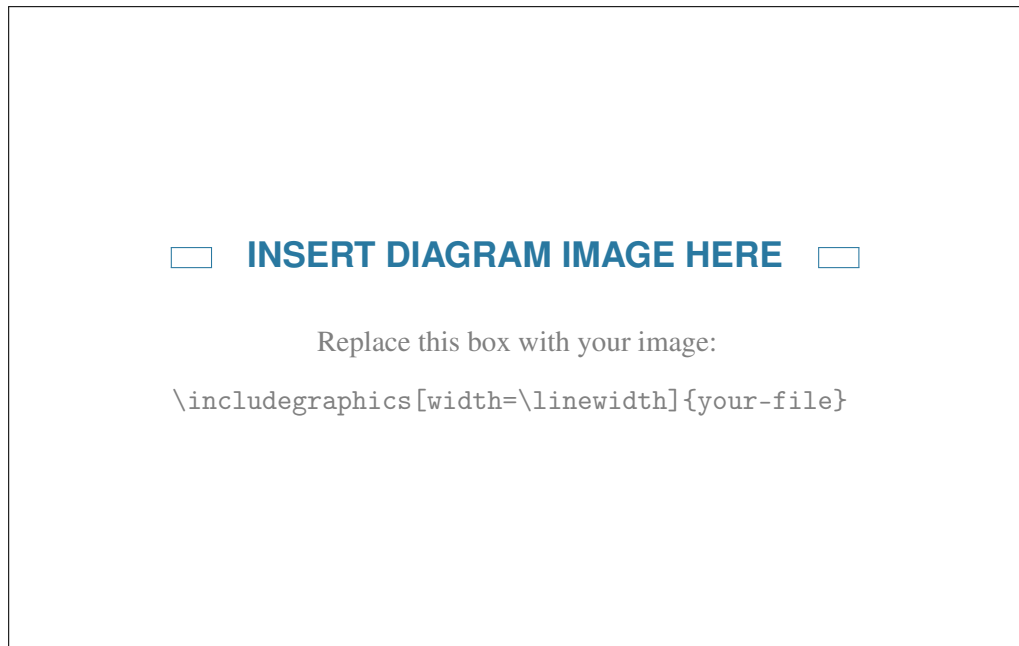


Figure 3.1. Use Case Diagram — VehicleWalay

Actors:

- **Buyer:** registers, searches via NLU, adds to watchlist, requests meetings, watches AI walkthroughs.
- **Dealer:** manages listings, confirms or rejects meeting requests, views analytics.
- **System (Scheduler):** auto-ingests listings every 6 hours, auto-flags fakes, dispatches alerts.

Use Cases: UC-01 Register/Login; UC-02 NLU Vehicle Search; UC-03 Voice Search; UC-04 AI Recommendations; UC-05 Add to Watchlist; UC-06 Receive Alerts; UC-07 Request Meeting; UC-08 Video Walkthrough; UC-09 Market Insights; UC-10 Report Suspicious Listing; UC-11 Manage Listings (Dealer); UC-12 Confirm Meeting; UC-13 View Analytics; UC-14 Auto-ingest Listings; UC-15 Flag Fake Listings; UC-16 Send Notifications.

Chapter 4

Software Design

4.1 Design Methodology

VehicleWalay adopts **Object-Oriented Design (OOD)** as its primary design methodology. The system is decomposed into clearly bounded classes representing core domain entities (*User, Vehicle, Dealer, ChatSession, Alert, Appointment*) with well-defined responsibilities and encapsulated state. OOD was selected because:

- The domain is entity-rich with persistent state (vehicles, users, watchlists, appointments).
- The React-based frontend and FastAPI/Pydantic backend both align naturally with object-oriented abstractions.
- Class, sequence, and state diagrams required by the FYP-1 rubric map directly to OOD artefacts.

4.2 Architectural Style

The system employs a **Layered Architecture** with a Service-Oriented decomposition of the backend, augmented by an **Event-Driven** notification subsystem. The primary layers are:

- **Presentation Layer:** Next.js 14 SPA; communicates via REST/JSON.
- **API Gateway Layer:** FastAPI router with JWT middleware, rate limiting, and CORS.

- **Service Layer:** discrete Python service classes — SearchService, RecommendationService, FakeDetectionService, AlertService, SchedulingService.
- **AI Layer:** LangChain orchestration encapsulating the RAG pipeline, embedding generation, and LLM calls.
- **Data Layer:** MongoDB for persistent storage, Redis for session caching, FAISS/ChromaDB for vector similarity search.
- **Ingestion Layer:** independent Scrapy-based scheduler running as a separate Docker service.

4.3 Architectural Diagram

Figure 4.1 illustrates the end-to-end request flow from user input through all system layers to the final response. The diagram shows six distinct layers communicating via well-defined interfaces: the Next.js frontend sends HTTPS REST requests to the FastAPI gateway; the gateway delegates to service classes; services call the LangChain AI layer for NLU, embedding, RAG, and LLM processing; the AI layer reads from and writes to the data layer (MongoDB, Redis, FAISS); and a separate ingestion container populates MongoDB via the Scrapy scraper every six hours.

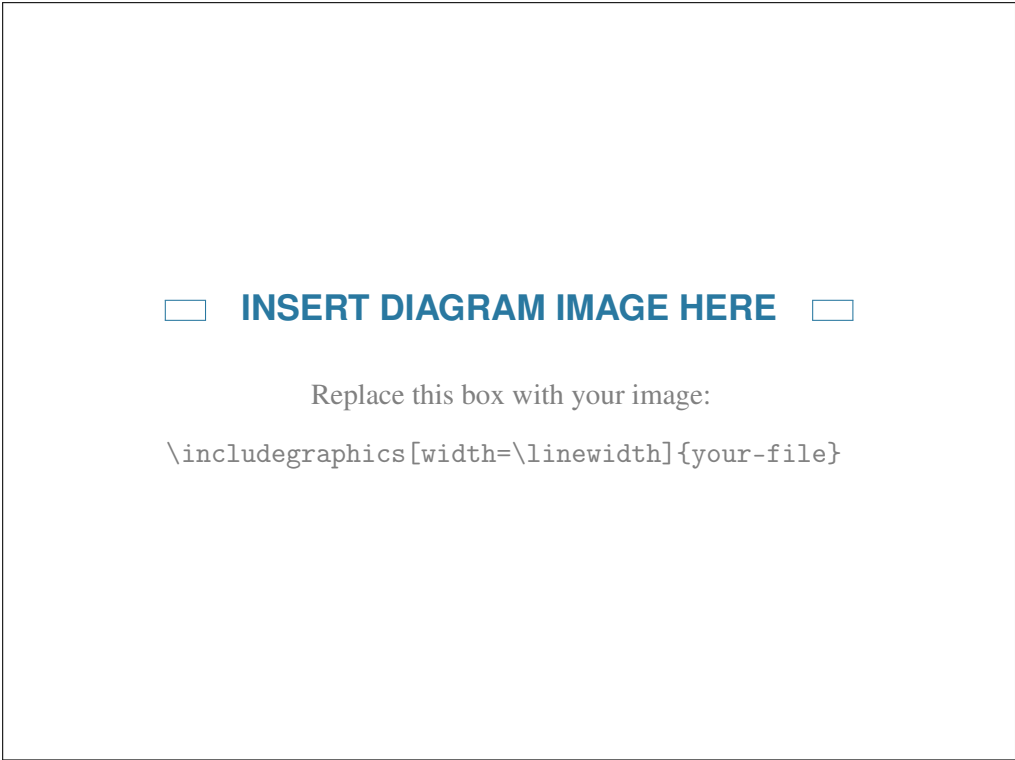


Figure 4.1. Architectural Diagram — VehicleWalay End-to-End System

4.4 Process Flow — Activity Diagram

Figure 4.2 traces the primary happy-path flow from a buyer submitting a natural-language query to receiving personalised vehicle recommendations and taking a follow-up action (watchlist or meeting request). Key decision points include: voice transcription before NLU, empty-result handling with relaxed parameters, re-ranking of retrieved listings by relevance and location, and the branching follow-up actions.



Figure 4.2. Activity Diagram — Vehicle Search and Recommendation Flow

4.5 Class Diagram

Figure 4.3 defines the primary domain classes, their attributes, methods, and relationships within the VehicleWalay system. The model includes seven core classes: *User*, *Vehicle*, *Dealer*, *Watchlist*, *ChatSession*, *Alert*, and *Appointment*.



Figure 4.3. Class Diagram — VehicleWalay Domain Model (OOD)

Key relationships:

- A *User* creates one *Watchlist* (1:1) and many *Alerts* (1:*).
- A *Dealer* lists many *Vehicles* (1:*).
- A *ChatSession* belongs to one *User* and can create one *Appointment*.
- An *Appointment* associates one Buyer, one Dealer, and one Vehicle.

4.6 Sequence Diagram — System Level

Figure 4.4 captures the system-level message flow for UC-02: a buyer submits a natural-language vehicle search query. The sequence spans six lifelines: Buyer, Frontend (Next.js), FastAPI, SearchService, AI Layer (LangChain), and MongoDB.



Figure 4.4. Sequence Diagram — UC-02: NLU Vehicle Search (System Level)

4.7 State Diagram

Figure 4.5 models the lifecycle of a **Vehicle Listing** entity within VehicleWalay. The listing passes through distinct states from ingestion to archival, with events and guard conditions controlling each transition.

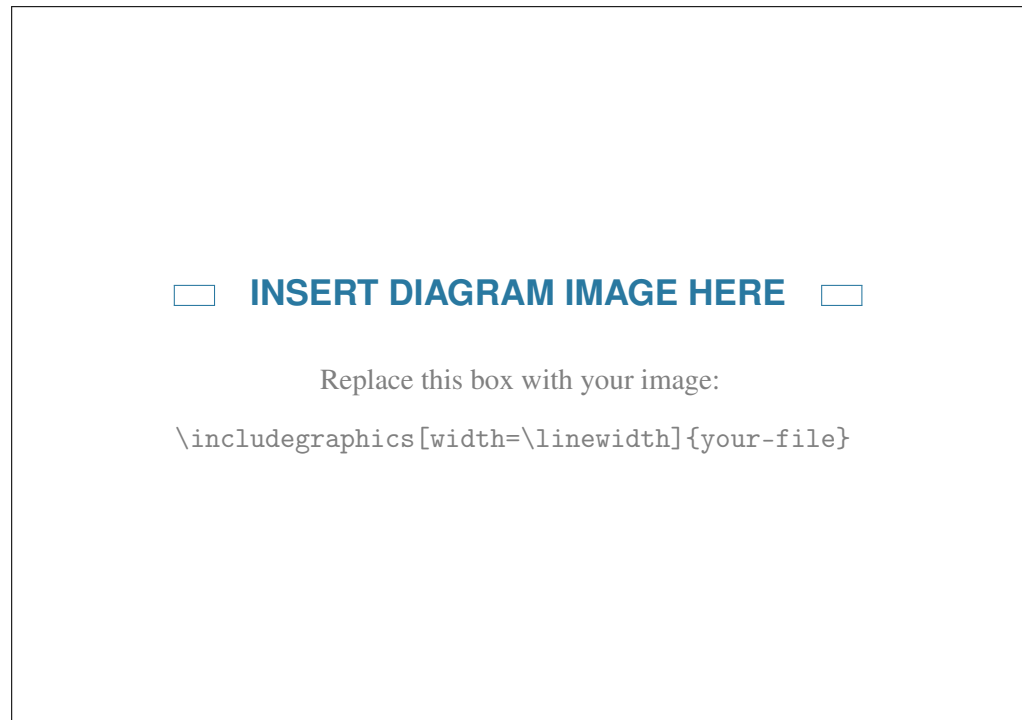


Figure 4.5. State Diagram — Vehicle Listing Lifecycle

States and transitions:

- **Ingested:** entry action — compute fake-score. *Guard:* fake-score ≥ 0.75 → **Flagged**; else → **Available**.
- **Flagged:** suppressed from default results. Event: dealer contest → **Under Review**.
- **Under Review:** admin verifies. approved → **Available**; rejected → **Removed**.
- **Available:** visible to buyers. price update → trigger Alert, stay **Available**; seller marks sold → **Sold**.
- **Watched:** in at least one user's watchlist. vehicle sold → **Sold**, notify all watching users.
- **Sold:** no further actions. After 30 days → **Archived**.
- **Archived:** read-only; excluded from all active queries.

4.8 Data Flow Diagram — Context Level (DFD Level 0)

Figure 4.6 shows the system as a single process, with all external entities and the data flows crossing the system boundary.

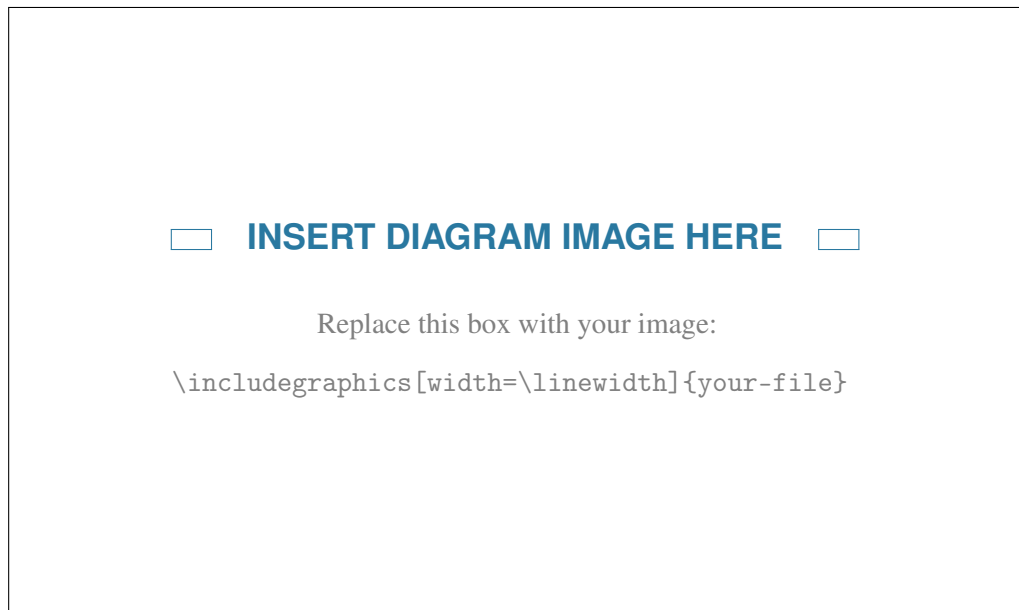


Figure 4.6. Data Flow Diagram — Context Level (Level 0)

External entities and data flows:

- **Buyer** → System: search query, watchlist actions, meeting requests.
- System → **Buyer**: vehicle results, recommendations, alerts, video walk-through, appointment confirmation.
- **Dealer** → System: listing data, availability slots, meeting responses.
- System → **Dealer**: enquiry summaries, analytics report.
- **PakWheels / OLX** → System: scraped listing data (title, price, city, mileage, URL).
- **LLM Provider** → System: generated natural-language responses.

4.9 Data Flow Diagram — Level 1

Figure 4.7 decomposes the single VehicleWalay process into its primary sub-processes, data stores, and inter-process flows.

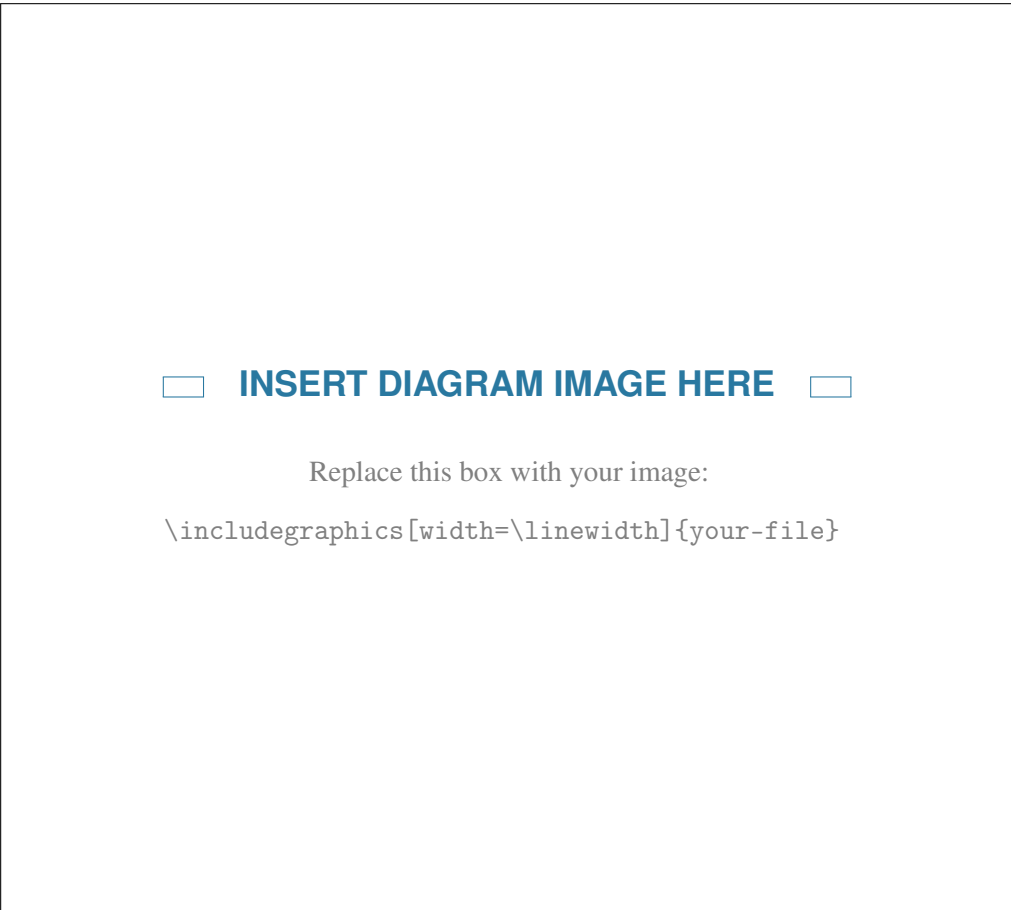


Figure 4.7. Data Flow Diagram — Level 1 Process Decomposition

Level-1 sub-processes:

1. **P1 — Authenticate User:** validates credentials, issues JWT token. DS: *DS1 Users*.
2. **P2 — Ingest & Score Listings:** scraper fetches raw listings; FakeDetection-Service computes fake-score. DS: *DS2 Vehicles, DS3 Vector Store*.
3. **P3 — Process Search Query (NLU + RAG):** embeds query; retrieves top-K from DS3; calls LLM; returns ranked results.
4. **P4 — Generate Recommendations:** reads user history from DS1; cosine

similarity against DS3.

5. **P5 — Manage Alerts:** monitors DS2 for price/status changes; dispatches notifications. DS: *DS4 Watchlists*, *DS5 Alerts*.
6. **P6 — Schedule Meetings:** buyer selects slot; writes appointment to DS6; notifies dealer.
7. **P7 — Generate AI Walkthrough:** fetches vehicle images from DS2; calls video generation API; returns URL.

4.10 Entity-Relationship Diagram

Figure 4.8 defines the persistent data model and relationships between all entities stored in MongoDB. The relational model is expressed for design clarity and is enforced via application-layer Pydantic schema validation.

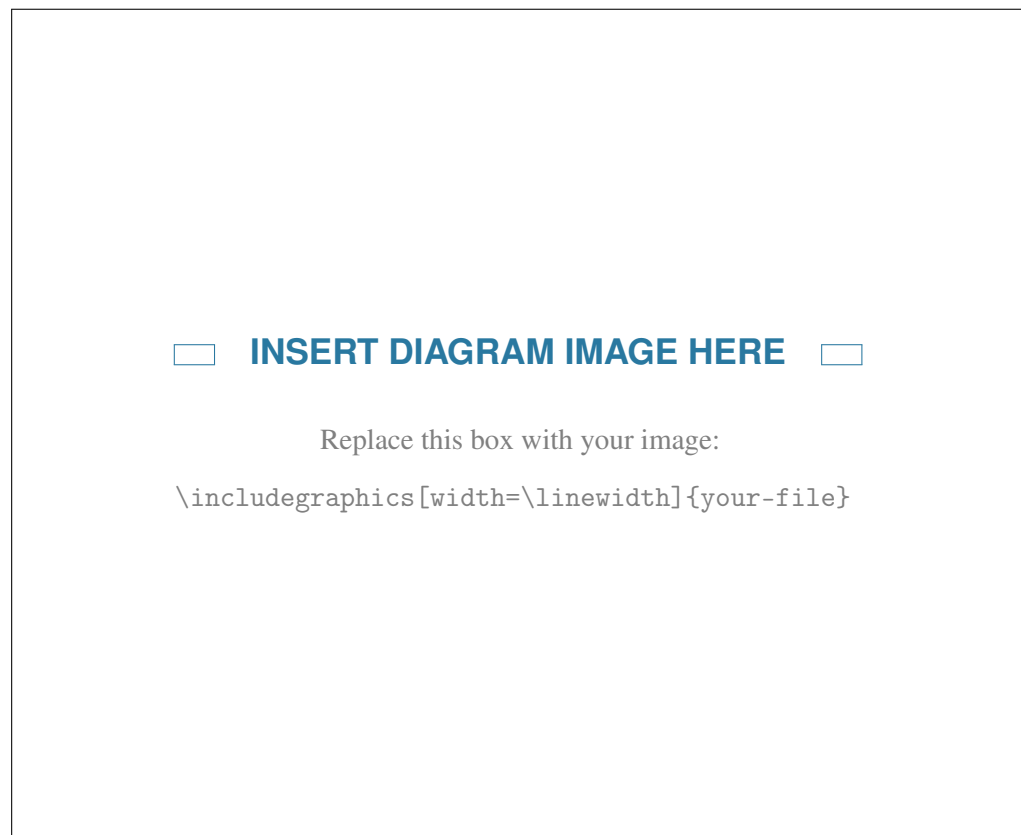


Figure 4.8. Entity-Relationship Diagram — VehicleWalay Data Model

Key entities and cardinalities:

- User (1) — has — (1) Watchlist
- User (1) — creates — (*) Alert
- Dealer (1) — lists — (*) Vehicle
- Watchlist (*) — references — (*) Vehicle
- Appointment — links — User, Dealer, Vehicle
- Vehicle (1) — has — (1) VectorEmbedding (FAISS/ChromaDB)

Chapter 5

Iteration Plan

Development is structured across four iterations aligned with the FYP-1 (February–May) and FYP-2 (September–December) academic calendar.

Table 5.1. Iteration Plan

Iter.	Period	Deliverables	Owner
1	Feb – Mar	Project setup, DB schema, scraping pipeline, auth API, basic chatbot UI	Bilal Aleem
2	Apr – May	NLU integration, RAG pipeline, recommendation engine, fake detection, trending insights	Both
3	Sep – Oct	Multilingual + voice, alert system, dealer dashboard, AI video generation	Both
4	Nov – Dec	Meeting automation agents, user history, end-to-end testing, deployment, final documentation	Both

Chapter 6

Ethical & Legal Considerations

- Data collection is limited to **publicly accessible** vehicle listing pages on PakWheels.com and OLX.com.pk. The scraper accesses only pages visible to unauthenticated visitors and stores listing metadata (title, price, city, URL) without capturing any personal user data from those platforms.
- No personally identifiable information (PII) of third-party platform users is stored. The system does **not** harvest seller phone numbers, email addresses, or private contact details.
- VehicleWalay users are presented with a clear **consent notice** at registration, explaining what data is collected (search history, watchlist preferences) and how it is used.
- **Notification opt-in is explicit and revocable.** Users may disable all alerts from their account settings at any time.
- The fake-listing flagging system is probabilistic, not deterministic. Flagged listings remain accessible via an explicit filter, and dealers may contest a flag via a support channel.
- The project complies with the spirit of Pakistan’s *Prevention of Electronic Crimes Act (PECA) 2016* with respect to data privacy and responsible scraping of public information.

Chapter 7

Comparison with Existing Systems

Table 7.1. Feature Comparison: VehicleWalay vs. Existing Platforms

Feature	PakWheel	OLX	Facebook	Dealers	VehicleWalay
NLU / AI-Powered Search	✗	✗	✗	✗	✓
Real-Time Listings	✓	✓	✓	✓	✓
Price Change / Sold Alerts	✓	✗	✓	✗	✓
Fake Listing Detection	✗	✗	✗	✓	✓
Multilingual (EN + Roman Urdu)	✗	✗	✗	✗	✓
AI Video Generation	✗	✗	✗	✗	✓
Buyer/Seller Meeting Automation	✗	✗	✗	✗	✓
AI Recommendation System	✗	✗	✗	✗	✓
User History & Search Recall	✗	✗	✗	✗	✓

References

- Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS, 2020.
- Chou, S., et al. (2021). *Fake Listing Detection in Peer-to-Peer Marketplaces*. ICWSM, 2021.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT, 2019.
- Karpukhin, V., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP, 2020.
- Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP, 2019.
- Winata, G., et al. (2021). *Are Multilingual Models Effective in Code-Switching?* EMNLP, 2021.
- Zhang, W., et al. (2019). *Deep Learning Based Recommender System: A Survey and New Perspectives*. ACM Computing Surveys, 52(1).
- PakWheels.com: <https://www.pakwheels.com/>
- OLX Motors Pakistan: <https://www.olx.com.pk/motors/>
- LangChain Documentation: <https://python.langchain.com/>
- Sentence Transformers: <https://www.sbert.net/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>